Laboratório Prolog · Aula 13

Rodrigo Del Padre · RA 24042092-2 · Turma B

Gerado em 19/10/2026, 20:25:12 · tempo registrado no cronômetro: 48:20

0 · Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

A consulta `paradigma(L, funcional)` unificará com o fato correspondente, respondendo `L = scheme`. A consulta `paradigma(java, logico)` responderá `false`, pois essa relação inexiste na base de conhecimento.

Saída do ola.pl

```
?- linguagem(_A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog, _A).
   paradigma(prolog, logico)
?- paradigma(_A, funcional).
   paradigma(scheme, funcional)
?- paradigma(java, logico).
   false.
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

`paradigma(prolog, P)` é uma consulta aberta com variável livre `P`: funciona como uma pergunta inquisitiva de busca ('Qual é o paradigma do Prolog?'), instanciando `P = logico`. Já `paradigma(prolog, logico)` é uma consulta fechada sem variáveis livres que funciona como uma asserção booleana ('É fato que Prolog é lógico?'), validando a prova afirmativamente.

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

Indica estritamente que a base do programa desconhece esse fato. O Prolog opera sob a Hipótese do Mundo Fechado (Closed World Assumption - CWA) e negação por falha: o que não pode ser deduzido dos fatos e regras fornecidos é considerado falso pelo motor de inferência.

---

1 · Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

`mulher(Quem)` terá 7 respostas. A conjunção `(mae(M, F), mulher(F))` filtrará apenas os pares mãe-filha, resultando em 4 pares: (lucia, tereza), (marta, joana), (tereza, bia) e (joana, ana).

Saída do familia.pl

```
--- 1. sem variavel: a resposta e true ou false
?- mae(joana, jake).
   mae(joana, jake)
?- mae(jake, joana).
   false.
?- pai(maria, jake).
   false.
--- 2. com variavel: o Prolog procura os valores que tornam a consulta verdadeira
?- mae(joana, _A).
   mae(joana, jake)
   mae(joana, ana)
?- pai(_A, jake).
   pai(jorge, jake)
?- mulher(_A).
   mulher(lucia)
   mulher(marta)
   mulher(tereza)
   mulher(joana)
   mulher(bia)
   mulher(ana)
   mulher(dora)
```

```
--- 3. conjuncao (virgula = E): as duas metas valem com o mesmo valor
?- mae(joana, _A), homem(_A).
   mae(joana, jake), homem(jake)
```

```
?- pai(vern, _A), mulher(_A).
   pai(vern, joana), mulher(joana)
?- mae(_A, _B), mulher(_B).
   mae(lucia, tereza), mulher(tereza)
   mae(marta, joana), mulher(joana)
   mae(tereza, bia), mulher(bia)
   mae(joana, ana), mulher(ana)
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

Significa: 'Encontre todos os casos onde alguém (Mae) é mãe de uma pessoa (Filha) e essa pessoa é uma mulher', mapeando todas as relações mãe-filha da árvore genealógica. Obteve exatamente 4 respostas.

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

Explica-se pela Hipótese do Mundo Fechado (CWA). Como `maria` não foi cadastrada, o Prolog assume falsidade automática. Em sistemas reais (como segurança ou medicina), isso é perigoso por equiparar 'ausência de registro' com 'negação factual absoluta'.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

Apareceram na ordem: lucia, marta, tereza, joana, bia, ana e dora. Essa sequência é determinada estritamente pela ordem física em que os fatos foram declarados no arquivo de código, pois a busca do Prolog opera sequencialmente de cima para baixo (top-down).

---

2 · Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

`tio(X, jake)` responderá X = paulo. `primo(leo, X)` responderá jake e ana.

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

Falharão `f(X, X) = f(a, b)`, `[A|B] = []` e `2 + 3 = 5`. Unificará apenas `f(X, X) = f(a, Y)` com X = a e Y = a.

Saídas de regras.pl e unificacao.pl

```
--- avo: duas regras progenitor encadeadas
?- avo(_A, jake).
   avo(lucia, jake)
   avo(osvaldo, jake)
?- avo(osvaldo, _A).
   avo(osvaldo, bia)
   avo(osvaldo, joana)
   avo(osvaldo, paulo)
--- irmao, tio e primo
?- irmao(jake, _A).
   irmao(jake, ana)
?- tio(_A, jake).
   tio(paulo, jake)
?- primo(leo, _A).
   primo(leo, jake)
   primo(leo, ana)
--- ancestral (recursiva)
?- ancestral(osvaldo, _A).
   ancestral(osvaldo, vern)
   ancestral(osvaldo, tereza)
   ancestral(osvaldo, bia)
   ancestral(osvaldo, joana)
   ancestral(osvaldo, paulo)
   ancestral(osvaldo, jake)
   ancestral(osvaldo, ana)
   ancestral(osvaldo, leo)
?- ancestral(_A, jake).
   ancestral(joana, jake)
   ancestral(jorge, jake)
   ancestral(marta, jake)
   ancestral(vern, jake)
   ancestral(lucia, jake)
   ancestral(osvaldo, jake)
?- ancestral(jake, _A).
   false.
```

```
--- constantes e variaveis
a  =  a    unifica
a  =  b    NAO unifica
_A  =  a    unifica   _A = a
--- estruturas
f(_A, b)  =  f(a, _B)    unifica   _A = a   _B = b
f(_A, _A)  =  f(a, b)    NAO unifica
f(_A, _A)  =  f(a, _B)    unifica   _A = a   _B = a
pai(_A, jake)  =  pai(jorge, _B)    unifica   _A = jorge   _B = jake
--- listas: [Cabeca|Cauda]
[_A|_B]  =  [1, 2, 3]    unifica   _A = 1   _B = [2, 3]
[_A|_B]  =  []    NAO unifica
[_A, _B|_C]  =  [a, b, c, d]    unifica   _A = a   _B = b   _C = [c, d]
--- aritmetica NAO e unificacao
2 + 3  =  5    NAO unifica
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

Porque expressa uma relação transitiva com profundidade genealógica variável. O caso base `ancestral(X, Y) :- progenitor(X, Y).` cobre os pais diretos. O caso recursivo `ancestral(X, Y) :- progenitor(X, Z), ancestral(Z, Y).` conecta sucessivas gerações intermediárias.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

A consulta passa a listar também `X = jake`. A restrição `X \== Y` é indispensável para evitar que o algoritmo classifique uma pessoa como irmã de si mesma por compartilharem os mesmos pais.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

Em `f(X, X) = f(a, b)`, X vincula-se com 'a' e colide com a constante 'b' no segundo termo. Em `f(X, X) = f(a, Y)`, X vincula-se com 'a' e unifica com a variável livre Y, resultando em `X = a` e `Y = a`.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Não é possível em linguagens funcionais ou imperativas, onde funções são unidirecionais ($y = f(x)$). Em Prolog, predicados definem relações lógicas multidirecionais e reversíveis, permitindo computar saídas a partir de entradas, entradas a partir de saídas ou testar proposições booleanas sem reescrever o algoritmo.

---

3 · Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

Responderá todas as decomposições contíguas possíveis em duas sublistas X e Y, totalizando 4 respostas: ([], [1,2,3]), ([1], [2,3]), ([1,2], [3]) e ([1,2,3], []).

Saída do listas.pl

```
--- membro: serve para testar E para gerar
?- membro(b, [a, b, c]).
   membro(b, [a, b, c])
?- membro(z, [a, b, c]).
   false.
?- membro(_A, [a, b, c]).
   membro(a, [a, b, c])
   membro(b, [a, b, c])
   membro(c, [a, b, c])
--- tamanho
?- tamanho([a, b, c], _A).
   tamanho([a, b, c], 3)
?- tamanho([], _A).
   tamanho([], 0)
--- concatena: o mesmo codigo, em varias direcoes
?- concatena([1, 2], [3, 4], _A).
   concatena([1, 2], [3, 4], [1, 2, 3, 4])
?- concatena(_A, [3], [1, 2, 3]).
   concatena([1, 2], [3], [1, 2, 3])
?- concatena([1], _A, [1, 2, 3]).
   concatena([1], [2, 3], [1, 2, 3])
?- concatena(_A, _B, [1, 2, 3]).
   concatena([], [1, 2, 3], [1, 2, 3])
   concatena([1], [2, 3], [1, 2, 3])
   concatena([1, 2], [3], [1, 2, 3])
   concatena([1, 2, 3], [], [1, 2, 3])
--- inverte
```

```
?- inverte([1, 2, 3], _A).
   inverte([1, 2, 3], [3, 2, 1])
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

O caso base é `tamanho([], 0).` e o recursivo é `tamanho([_|Cauda], N) :- tamanho(Cauda, M), N is M + 1.`. A cada invocação, a soma `N is M + 1` fica pendente na pilha, sendo avaliada somente no desempilhamento.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Porque existem 4 formas de dividir a lista em prefixo e sufixo. Em Scheme, `anexar` só une duas listas conhecidas; ela não executa particionamento inverso a partir do resultado final.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

Foi realizado por unificação de padrões e backtracking automático: a primeira cláusula casa a cabeça ('a'); no retrocesso, a segunda cláusula descarta a cabeça e relança o predicado sobre a cauda remanescente `[b, c]`.

---

4 · Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

`X = 2 + 3` unifica o termo simbólico `X = 2 + 3`. `X is 2 + 3` calcula `X = 5`. `fatorial(N, 120)` lançará `instantiation_error` por tentar avaliar expressões aritméticas com variáveis livres.

Saída do aritmetica.pl

```
--- = unifica, is calcula
?- _A = 2+3.
   2+3 = 2+3
?- _A is 2+3.
   5 is 2+3
?- 5=:=2+3.
   5=:=2+3
?- 2+3=5.
   false.
?- _A is 7 mod 3.
   1 is 7 mod 3
?- _A is 7//2.
   3 is 7//2
?- _A is 2^10.
   1024 is 2^10
--- fatorial, fib e soma
?- fatorial(5, _A).
   fatorial(5, 120)
?- fatorial(0, _A).
   fatorial(0, 1)
?- fib(10, _A).
   fib(10, 55)
?- soma([1, 2, 3, 4], _A).
   soma([1, 2, 3, 4], 10)
--- aritmetica nao e relacional
?- fatorial(_A, 120).
   ERRO: error(instantiation_error,context(>(/(_A,0),0),_B))
?- _A is _B+1.
   ERRO: error(instantiation_error,context(is/2,_A))
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

`=` unifica termos estruturais simbolicamente (`X = 2 + 3` gera o termo `+(2,3)`). `is` calcula o lado direito e unifica com a esquerda (`X is 2 + 3` gera `X = 5`). `=:=` avalia algebricamente os dois lados e compara a igualdade de valores (`2 + 3 =:= 4 + 1` é `true`).

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

Porque manipulações de listas dependem puramente de casamento estrutural de termos, enquanto a aritmética procedural via `is` e operadores relacionais exige variáveis concretas já instanciadas, não operando como resolvedor algébrico bidirecional.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Ambos usam raciocínio indutivo com caso base e passo recursivo. Em Haskell, trata-se de uma função matemática pura $Int o Int$; em Prolog, é uma relação predicativa com unificação explícita do acumulador via operador `is`.

5 · Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

`solteiro2(X)` resultará em false. `maximo2(5, 3, M)` produzirá 2 respostas concorrentes. `maximo3(5, 3, 3)` retornará true erroneamente.

Saída de controle.pl e mensagem de ancestral_ruim

```
--- A) backtracking: acompanhe a ordem das tentativas
?- mesmo_gosto(X, Y).
   tenta ana e ana
   tenta ana e bia
   tenta ana e caio
   resposta: mesmo_gosto(ana, caio)
   tenta bia e ana
   tenta bia e bia
   tenta bia e caio
   tenta caio e ana
   resposta: mesmo_gosto(caio, ana)
   tenta caio e bia
   tenta caio e caio
--- B) negacao: a ordem das metas muda a resposta
?- solteiro1(_A).
   solteiro1(osvaldo)
   solteiro1(paulo)
   solteiro1(jake)
?- solteiro2(_A).
   false.
--- C) corte
?- maximo1(5, 3, _A).
   maximo1(5, 3, 5)
?- maximo2(5, 3, _A).
   maximo2(5, 3, 5)
   maximo2(5, 3, 3)
?- maximo3(5, 3, _A).
   maximo3(5, 3, 5)
?- maximo1(5, 3, 3).
   false.
?- maximo3(5, 3, 3).
   maximo3(5, 3, 3)

ancestral_bom(osvaldo, jake)?
  sim
ancestral_ruim(osvaldo, jake)? (vai estourar a pilha)
Fatal Error: local stack overflow
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

Existem 24 colorações viáveis com 3 cores e 0 com 2 cores, devido à adjacência mútua triangular entre PR, SP e MS que demanda pelo menos 3 cores cromáticas.

Saída do mapa.pl

```
--- coloracoes com 3 cores, na ordem [RS, SC, PR, SP, MS, MG, RJ, ES]
[vermelho, verde, azul, vermelho, verde, azul, verde, vermelho]
[vermelho, verde, azul, vermelho, verde, azul, vermelho, verde]
--- quantas coloracoes existem com 3 cores?
24
--- e com 2 cores?
?- mapa([vermelho, verde], _A).
   false.
--- e com 4 cores?
2160
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

Na tentativa `ana` e `ana`, a desigualdade `X \== Y` falhou; o interpretador fez backtracking retornando a `amigo(Y)` e testou `bia`. Em seguida, a meta de preferência de bebida falhou (café vs chá); novo retrocesso escolheu `caio`, satisfazendo plenamente a conjunção e emitindo a resposta. O ciclo prosseguiu até explorar todos os pares.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

Em `solteiro2`, a negação `\+ casado(X)` roda com X livre; como existem casados na base, `casado(X)` é verdadeiro e sua negação falha imediatamente. Em `solteiro1`, a meta `homem(X)` primeiro liga X a indivíduos específicos antes do teste de negação. `osvaldo` é qualificado como solteiro pela Hipótese do Mundo Fechado, pois seu casamento não consta explicitado nos fatos.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

O corte não atuou porque a unificação no cabeçalho da primeira cláusula `maximo3(X, Y, X)` falhou ao casar o terceiro argumento (3) com o primeiro (5), abortando a regra antes de atingir o corte `!`. O Prolog então avançou para a segunda cláusula `maximo3(_, Y, Y)`, aceitando 3 como valor. Corrige-se deslocando a unificação para depois do corte: `maximo(X, Y, M) :- X >= Y, !, M = X. maximo(_, Y, Y).`.

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

Porque o Prolog resolve metas estritamente da esquerda para a direita. Em `ancestral_ruim`, a chamada recursiva à esquerda invoca a si mesma indefinidamente sem consumir nenhum predicado concreto, provocando recursão infinita e estouro de pilha. Em `ancestral_bom`, consome-se primeiro o fato `progenitor(X, Z)` antes de recursar sobre Z, garantindo convergência.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

Foram identificadas 24 soluções com 3 cores, 0 com 2 cores e 2160 com 4 cores. O programa é declarativo porque expressa apenas as restrições e regras do domínio (estados vizinhos devem ter cores diferentes), transferindo a complexidade de busca e backtracking para o motor de inferência.

---

6 · O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No paradigma funcional (`anexar`), o código é estruturado como funções matemáticas unidirecionais que mapeiam entradas preestabelecidas em uma saída calculada. No paradigma lógico (`concatena`), o código expressa relações predicativas multidirecionais e reversíveis: uma mesma relação serve para unir listas, particionar coleções ou validar divisões via unificação e retrocesso.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

1) Sensibilidade extrema à ordem das cláusulas e metas (recursão à esquerda causando estouro de pilha em `laco_infinito.pl` / `ancestral_ruim`).
2) Perda de reversibilidade relacional no cálculo numérico (`instantiation_error` ao avaliar `fatorial(N, 120)` em `aritmetica.pl`).

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog para sistemas especialistas baseados em regras, verificação formal de teoremas, análise de linguagem natural (gramáticas DCG) e problemas de satisfação de restrições (puzzles combinatórios). Não escolheria Prolog para sistemas corporativos com alta concorrência assíncrona, microsserviços REST, manipulação numérica de matrizes ou interfaces gráficas pesadas, onde o ecossistema robusto do Java é amplamente superior.

Exercícios opcionais

Opcionais: resultado dos testes

```
Exercicio 1
   ok      irma(ana, jake)
   ok      irma(tereza, vern)
   ok      NAO irma(jake, ana)
   ok      tia(tereza, joana)
   ok      tia(joana, leo)
   ok      NAO tia(joana, jake)
Exercicio 2
   ok      ultimo([a,b,c], c)
   ok      ultimo([7], 7)
   ok      NAO ultimo([a,b,c], b)
Exercicio 3
   ok      soma_quadrados([1,2,3], 14)
   ok      soma_quadrados([], 0)
   ok      pares([1,2,3,4,5,6], [2,4,6])
   ok      pares([1,3], [])
Exercicio 4
   ok      mdc(12, 18, 6)
   ok      mdc(17, 5, 1)
```

```
   ok      mdc(10, 0, 10)
   ok      potencia(2, 10, 1024)
   ok      potencia(5, 0, 1)
Exercicio 5
   ok      caminho(curitiba, foz)
   ok      caminho(joinville, cascavel)
   ok      NAO caminho(foz, curitiba)
   ok      NAO caminho(curitiba, paris)
Exercicio 6
   ok      3 cores: 12 coloracoes
   ok      2 cores: 2 coloracoes
   ok      1 cor: nenhuma
Exercicio 7
   ok      colega(ana, bruno)
   ok      NAO colega(ana, carla)
   ok      NAO colega(ana, ana)
   ok      ganha_mais(ana, bruno)
   ok      NAO ganha_mais(elisa, ana)
   ok      total_depto(ti, 12500)
   ok      total_depto(rh, 9000)
```

E1 a E7 Códigos implementados:

```
% E1: irma e tia
irma(X, Y) :- mulher(X), irmao(X, Y).
tia(X, Y) :- irma(X, P), progenitor(P, Y).

% E2: ultimo
ultimo([X], X).
ultimo([_|Cauda], X) :- ultimo(Cauda, X).

% E3: funcoes funcionais
soma_quadrados([], 0).
soma_quadrados([H|T], S) :- soma_quadrados(T, S1), S is S1 + (H * H).
pares([], []).
pares([H|T], [H|P]) :- H mod 2 =:= 0, !, pares(T, P).
pares([_|T], P) :- pares(T, P).

% E4: mdc e potencia
mdc(A, 0, A) :- A > 0, !.
mdc(A, B, D) :- B > 0, R is A mod B, mdc(B, R, D).
potencia(_, 0, 1) :- !.
potencia(Base, Exp, R) :- Exp > 0, Exp1 is Exp - 1, potencia(Base, Exp1, R1), R is R1 * Base.

% E5: caminho em grafo com ciclos
caminho(A, B) :- caminho(A, B, [A]).
caminho(A, B, _) :- liga(A, B).
caminho(A, B, Vis) :- liga(A, C), \+ membro(C, Vis), caminho(C, B, [C|Vis]).

% E6: mapa menor
mapa_sul(Paleta, [RS, SC, PR]) :-
    membro(RS, Paleta), membro(SC, Paleta), membro(PR, Paleta),
    RS \== SC, SC \== PR.

% E7: banco de dados
colega(X, Y) :- funcionario(X, D, _), funcionario(Y, D, _), X \== Y.
ganha_mais(X, Y) :- funcionario(X, _, S1), funcionario(Y, _, S2), S1 > S2.
total_depto(D, Total) :- findall(Sal, funcionario(_, D, Sal), Lista), soma(Lista, Total).
```